

FRD

Functional Requirements Document (FRD)

AWS Glue Data Pipeline – Customer Order Processing

•

1. Document Overview

Purpose

This Functional Requirements Document defines the functional behavior of an AWS Glue-based data pipeline that ingests customer and order data from S3, performs data quality operations, implements SCD Type 2 logic using Apache Hudi, and produces analytical aggregates for business intelligence purposes.

Scope

This FRD covers:

- Ingestion of customer and order CSV files from S3
- Data cleaning and deduplication
- Registration of datasets in AWS Glue Data Catalog
- Implementation of Slowly Changing Dimension Type 2 (SCD2) logic for order summary
- Incremental processing triggered by customer data changes
- Generation of customer aggregate spend analytics
- Use of AWS services: S3, Glue, Athena, CloudWatch, and optionally Lake Formation

Out of scope:

- Real-time streaming ingestion
- Data visualization or BI tool integration
- User interface development
- Authentication and authorization mechanisms (beyond AWS IAM)

Assumptions

- CSV files are well-formed and use standard delimiters
- Source files arrive in designated S3 paths with consistent naming conventions
- AWS Glue job execution environment has appropriate IAM permissions for S3, Glue Catalog, and CloudWatch
- Hudi library is available in the Glue job runtime
- "Null" (string) and NULL (actual null) are the only null representations requiring removal
- Date fields in source data are in a parseable format

- CustId is the primary key for Customer data
- OrderId is the primary key for Order data
- The term "change" in customer data refers to any modification in Name, EmailId, or Region fields
- OpTs (operation timestamp) is system-generated at processing time
- StartDate and EndDate for SCD2 are system-managed timestamps
- IsActive flag uses boolean or integer (1/0) representation
-

2. Functional Requirements

FR-INGEST-001: Ingest Customer Data from S3

- Title: Load Customer CSV Files from S3
- Description:

The system shall read customer data in CSV format from the specified S3 location and load it into a Glue DynamicFrame for processing.

- Business Objective:

Enable downstream processing of customer master data for order enrichment and analytics.

- Inputs:
 - Source: s3://adif-sdlc/sdlc_wizard/customerdata/
 - Format: CSV
 - Schema:
 - CustId: (type not specified in BRD)
 - Name: (type not specified in BRD)
 - EmailId: (type not specified in BRD)
 - Region: (type not specified in BRD)
- Processing Rules / Functional Steps:
 1. Connect to S3 path s3://adif-sdlc/sdlc_wizard/customerdata/
 2. Read all CSV files present in the path
 3. Parse CSV with header row
 4. Load data into Glue DynamicFrame
- Outputs:
 - In-memory DynamicFrame containing raw customer records
- Preconditions:

- S3 bucket and path exist and are accessible
- CSV files contain header row matching schema
- Glue job has read permissions on the S3 path
- Postconditions:
- Customer data is available in memory for cleaning and transformation
- Error / Validation Conditions:
- If S3 path does not exist, job shall fail with clear error message
- If CSV parsing fails, job shall log error and halt processing
- If no files are found in the path, job shall log warning and exit gracefully
-

FR-INGEST-002: Ingest Order Data from S3

- Title: Load Order CSV Files from S3
- Description:

The system shall read order transaction data in CSV format from the specified S3 location and load it into a Glue DynamicFrame for processing.

- Business Objective:

Enable processing of order transactions for customer spend analysis and historical tracking.

- Inputs:
- Source: s3://adif-sdlc/sdlc_wizard/orderdata/
- Format: CSV
- Schema:
- OrderId: (type not specified in BRD)
- ItemName: (type not specified in BRD)
- PricePerUnit: (type not specified in BRD)
- Qty: (type not specified in BRD)
- Date: (type not specified in BRD)
- CustId: (type not specified in BRD)
- Processing Rules / Functional Steps:
- 1. Connect to S3 path s3://adif-sdlc/sdlc_wizard/orderdata/
- 2. Read all CSV files present in the path
- 3. Parse CSV with header row

4. Load data into Glue DynamicFrame

- Outputs:
- In-memory DynamicFrame containing raw order records
- Preconditions:
- S3 bucket and path exist and are accessible
- CSV files contain header row matching schema
- Glue job has read permissions on the S3 path
- Postconditions:
- Order data is available in memory for cleaning and transformation
- Error / Validation Conditions:
- If S3 path does not exist, job shall fail with clear error message
- If CSV parsing fails, job shall log error and halt processing
- If no files are found in the path, job shall log warning and exit gracefully
-

FR-CLEAN-001: Remove Null Values and Duplicates from Customer Data

- Title: Data Quality Processing for Customer Dataset
- Description:

The system shall cleanse customer data by removing records containing null values (both string "Null" and actual NULL) and eliminating duplicate records.

- Business Objective:

Ensure data quality and consistency in the customer master dataset to prevent downstream processing errors and inaccurate analytics.

- Inputs:
- Raw customer DynamicFrame from FR-INGEST-001
- Processing Rules / Functional Steps:
- 1. Identify and remove rows where any field contains the string "Null" (case-sensitive)
- 2. Identify and remove rows where any field contains actual NULL values
- 3. Remove duplicate rows based on all columns
- 4. Retain only unique, non-null customer records
- Outputs:
- Cleaned customer DynamicFrame ready for catalog registration

- Preconditions:
- Customer data has been successfully ingested per FR-INGEST-001
- Postconditions:
- Customer dataset contains no null values or duplicates
- Record count is logged for audit purposes
- Error / Validation Conditions:
- If all records are removed during cleaning, log warning and halt downstream processing
- Log count of removed null records and duplicate records
-

FR-CLEAN-002: Remove Null Values and Duplicates from Order Data

- Title: Data Quality Processing for Order Dataset
- Description:

The system shall cleanse order data by removing records containing null values (both string "Null" and actual NULL) and eliminating duplicate records.

- Business Objective:

Ensure data quality and consistency in the order transaction dataset to enable accurate spend calculations and historical tracking.

- Inputs:
- Raw order DataFrame from FR-INGEST-002
- Processing Rules / Functional Steps:
 1. Identify and remove rows where any field contains the string "Null" (case-sensitive)
 2. Identify and remove rows where any field contains actual NULL values
 3. Remove duplicate rows based on all columns
 4. Retain only unique, non-null order records
- Outputs:
- Cleaned order DataFrame ready for catalog registration
- Preconditions:
- Order data has been successfully ingested per FR-INGEST-002
- Postconditions:
- Order dataset contains no null values or duplicates

- Record count is logged for audit purposes
- Error / Validation Conditions:
- If all records are removed during cleaning, log warning and halt downstream processing
- Log count of removed null records and duplicate records
-

FR-CATALOG-001: Register Customer Table in Glue Data Catalog

- Title: Create Glue Catalog Table for Customer Data
- Description:

The system shall register the cleaned customer dataset as a table in the AWS Glue Data Catalog under the specified database.

- Business Objective:

Enable SQL-based querying of customer data via Athena and provide metadata management for the data lake.

- Inputs:
- Cleaned customer DynamicFrame from FR-CLEAN-001
- Target Database: gen_ai_poc_databricksco
- Target Table: sdlc_wizard_customer
- Processing Rules / Functional Steps:
 1. Write cleaned customer data to Glue Data Catalog
 2. Create or update table sdlc_wizard_customer in database gen_ai_poc_databricksco
 3. Infer and register schema from DynamicFrame
 4. Enable table for Athena queries
- Outputs:
- Glue Catalog table: gen_ai_poc_databricksco.sdlc_wizard_customer
- Table metadata registered and queryable
- Preconditions:
- Database gen_ai_poc_databricksco exists in Glue Data Catalog
- Glue job has permissions to create/update catalog tables
- Customer data has been cleaned per FR-CLEAN-001
- Postconditions:
- Customer table is available for querying via Athena

- Table schema is documented in Glue Catalog
- Error / Validation Conditions:
- If database does not exist, job shall fail with clear error message
- If table creation fails, log error and halt processing
-

FR-CATALOG-002: Register Order Table in Glue Data Catalog

- Title: Create Glue Catalog Table for Order Data
- Description:

The system shall register the cleaned order dataset as a table in the AWS Glue Data Catalog under the specified database.

- Business Objective:

Enable SQL-based querying of order transaction data via Athena and provide metadata management for the data lake.

- Inputs:
- Cleaned order DynamicFrame from FR-CLEAN-002
- Target Database: gen_ai_poc_databricksco
- Target Table: sdlc_wizard_order
- Processing Rules / Functional Steps:
 1. Write cleaned order data to Glue Data Catalog
 2. Create or update table sdlc_wizard_order in database gen_ai_poc_databricksco
 3. Infer and register schema from DynamicFrame
 4. Enable table for Athena queries
- Outputs:
- Glue Catalog table: gen_ai_poc_databricksco.sdlc_wizard_order
- Table metadata registered and queryable
- Preconditions:
- Database gen_ai_poc_databricksco exists in Glue Data Catalog
- Glue job has permissions to create/update catalog tables
- Order data has been cleaned per FR-CLEAN-002
- Postconditions:
- Order table is available for querying via Athena

- Table schema is documented in Glue Catalog
- Error / Validation Conditions:
- If database does not exist, job shall fail with clear error message
- If table creation fails, log error and halt processing
-

FR-SCD2-001: Implement SCD Type 2 for Order Summary

- Title: Maintain Historical Order Summary with SCD Type 2 Logic
- Description:

The system shall join customer and order data, implement Slowly Changing Dimension Type 2 logic using Apache Hudi, and maintain historical versions of order summary records with active/inactive flags and effective date ranges.

- Business Objective:

Preserve complete historical lineage of customer-order relationships to support time-based analytics, auditing, and regulatory compliance.

- Inputs:
 - Cleaned customer data from FR-CLEAN-001
 - Cleaned order data from FR-CLEAN-002
 - Target Database: gen_ai_poc_databrickscoe
 - Target Table: ordersummary
 - Target S3 Location: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
 - Processing Rules / Functional Steps:
1. Join customer and order datasets on CustId
 2. For each joined record, check if a matching active record exists in ordersummary (based on business key)
 3. If no active record exists, insert new record with:
 - IsActive = True
 - StartDate = current timestamp
 - EndDate = null
 - OpTs = current timestamp
 4. If active record exists and customer attributes have changed:
 - Update existing active record: set IsActive = False, EndDate = current timestamp
 - Insert new record with updated values: IsActive = True, StartDate = current timestamp, EndDate = null, OpTs = current timestamp

5. If active record exists and no change detected, skip processing
6. Write results to S3 using Hudi format with upsert operation
7. Register/update ordersummary table in Glue Catalog

- Outputs:

- Hudi table at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Glue Catalog table: gen_ai_poc_databrickscoe.ordersummary
- Records with SCD2 attributes: IsActive, StartDate, EndDate, OpTs

- Preconditions:

- Customer and order data have been cleaned and are available
- Hudi library is available in Glue job environment
- Target S3 path is accessible with write permissions
- Database gen_ai_poc_databrickscoe exists

- Postconditions:

- All current customer-order relationships are marked as active
- Historical versions of changed records are preserved with proper date ranges
- Table is queryable via Athena with time-travel capability

- Error / Validation Conditions:

- If join produces no records, log warning and skip Hudi write
- If Hudi write fails, job shall fail with error details
- If customer data contains orphaned orders (CustId not in customer table), log warning and exclude from processing

-

FR-INCR-001: Incremental Processing Based on Customer Changes

- Title: Trigger SCD2 Updates on Customer Data Changes
- Description:

The system shall detect changes in customer data and trigger incremental updates to the ordersummary table, ensuring that only affected records are reprocessed.

- Business Objective:

Optimize processing performance by updating only changed records rather than full reprocessing, while maintaining data consistency and historical accuracy.

- Inputs:

- New/updated customer data from FR-INGEST-001

- Existing ordersummary Hudi table from FR-SCD2-001
- Processing Rules / Functional Steps:
 1. Identify customer records that have changed (new CustId or modified Name, EmailId, or Region)
 2. For changed customers, retrieve all associated order records
 3. Apply SCD2 logic per FR-SCD2-001 only for affected customer-order combinations
 4. Update ordersummary table incrementally using Hudi upsert
 5. Maintain IsActive, StartDate, EndDate, and OpTs fields consistently
- Outputs:
 - Incrementally updated ordersummary Hudi table
 - Updated Glue Catalog metadata
- Preconditions:
 - Ordersummary table exists from previous run (FR-SCD2-001)
 - Customer change detection mechanism is in place
 - Order data is available for affected customers
- Postconditions:
 - Only changed customer-order records are updated in ordersummary
 - Historical integrity is maintained
 - Processing time is optimized compared to full refresh
- Error / Validation Conditions:
 - If no customer changes detected, log info message and skip processing
 - If change detection fails, fall back to full processing per FR-SCD2-001
 - Log count of affected customers and updated records
-

FR-AGG-001: Generate Customer Aggregate Spend Table

- Title: Create Customer Total Spend Analytics Table
- Description:

The system shall aggregate order data by customer to calculate total spending and produce an analytics-ready table for business intelligence purposes.

- Business Objective:

Provide business users with summarized customer spending metrics for customer segmentation, marketing campaigns, and revenue analysis.

- Inputs:
- Cleaned customer data from FR-CLEAN-001
- Cleaned order data from FR-CLEAN-002
- Target Database: gen_ai_poc_databricksco
- Target Table: customeraggregatespend
- Target S3 Location: s3://adif-sdlc/analytics/customeraggregatespend/
- Processing Rules / Functional Steps:
 1. Join customer and order data on CustId
 2. Calculate TotalAmount for each customer: SUM(PricePerUnit Qty)
 3. Group by customer Name and Date
 4. Select columns: Name, TotalAmount, Date
 5. Write aggregated results to S3 location s3://adif-sdlc/analytics/customeraggregatespend/
 6. Register/update customeraggregatespend table in Glue Catalog
- Outputs:
- Analytics table at s3://adif-sdlc/analytics/customeraggregatespend/
- Glue Catalog table: gen_ai_poc_databricksco.customeraggregatespend
- Schema: Name, TotalAmount, Date
- Preconditions:
 - Customer and order data have been cleaned per FR-CLEAN-001 and FR-CLEAN-002
 - Target S3 path is accessible with write permissions
 - Database gen_ai_poc_databricksco exists
- Postconditions:
 - Customer aggregate spend data is available for querying via Athena
 - Table contains one row per customer per date with total spending
 - Data is optimized for BI tool consumption
- Error / Validation Conditions:
 - If join produces no records, log warning and create empty table
 - If aggregation fails, job shall fail with error details
 - Validate that TotalAmount is non-negative; log warning for negative values
-

3. Non-Functional Considerations

Monitoring and Observability

- All Glue jobs shall log execution metrics to AWS CloudWatch
- Job success/failure status shall be tracked
- Record counts at each processing stage shall be logged
- Processing duration shall be monitored

Data Governance (Optional)

- AWS Lake Formation may be used for fine-grained access control
- Data catalog tables may be registered with Lake Formation for governance policies

Query Performance

- Output tables shall be optimized for Athena query performance
- Hudi table format enables time-travel queries and incremental reads

Scalability

- Solution shall leverage AWS Glue auto-scaling capabilities
- S3 storage shall accommodate growing data volumes
- Hudi format supports efficient upserts for large datasets
-

4. Requirement Traceability Notes

Requirement ID	BRD Section	Description
-----	-----	-----
FR-INGEST-001	Source Input (AWS S3) - Customer CSV	Maps to customer data ingestion from s3://adif-sdlc/sdlc_wizard/customerdata/
FR-INGEST-002	Source Input (AWS S3) - Order CSV	Maps to order data ingestion from s3://adif-sdlc/sdlc_wizard/orderdata/
FR-CLEAN-001	Data Cleaning	Implements null and duplicate removal for customer data
FR-CLEAN-002	Data Cleaning	Implements null and duplicate removal for order data
FR-CATALOG-001	Glue Catalog Tables	Creates sdlc_wizard_customer table in gen_ai_poc_databrickscoe
FR-CATALOG-002	Glue Catalog Tables	Creates sdlc_wizard_order table in gen_ai_poc_databrickscoe
FR-SCD2-001	SCD Type 2 (Hudi on S3)	Implements SCD2 logic for ordersummary table with Hudi
FR-INCR-001	Incremental Logic	Handles customer update triggers for SCD2 processing
FR-AGG-001	Aggregation Table	Creates customeraggregatespend analytics table

All requirements trace directly to explicit BRD sections. No functional behavior has been invented beyond what is stated in the BRD.

-
- End of Functional Requirements Document